Packet Sniffing and Spoofing Lab Report

Students: İsmail Babacan & Veysel Reşit Çaçan

Course: Information System Security

Lab: SEED Labs — Packet Sniffing and Spoofing

1. Introduction

In this lab, we studied two important network security threats: **packet sniffing** and **packet spoofing**. The goal was not only to use existing tools but also to understand how these techniques work at a technical level by writing our own programs.

Packet sniffing means capturing packets that travel on a network. An attacker who is on the same network can use a sniffer program to read other people's traffic. Packet spoofing means sending a packet with a fake source IP address. The receiver cannot understand that the packet did not come from a real source.

We worked with two different approaches. In the first part, we used **Scapy**, which is a Python library that makes it easy to create and capture packets. In the second part, we wrote our own programs in **C** using the pcap library and raw sockets. This helped us understand what happens at a lower level.

The lab environment had three machines connected on the same LAN (10.9.0.0/24):

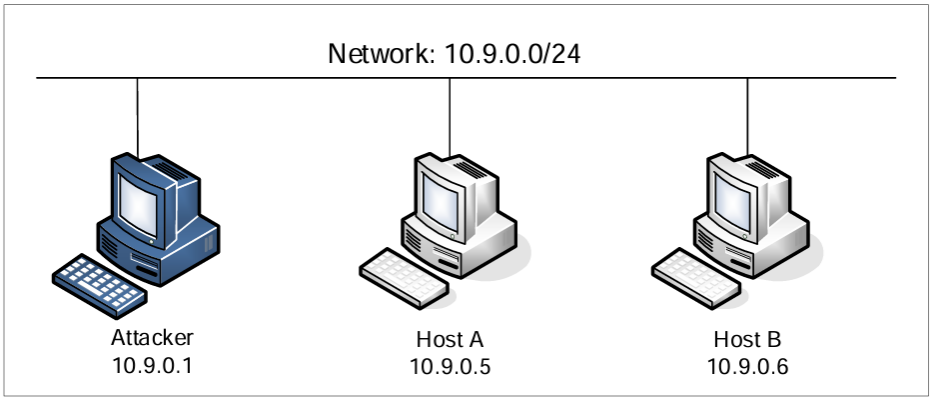


Figure 1: Lab environment setup

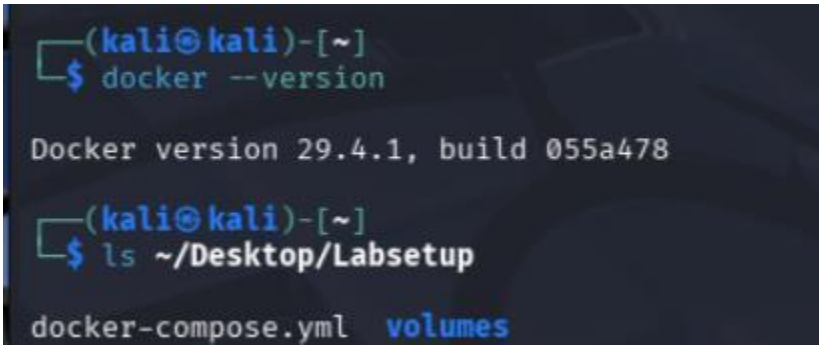
2. Environment Setup

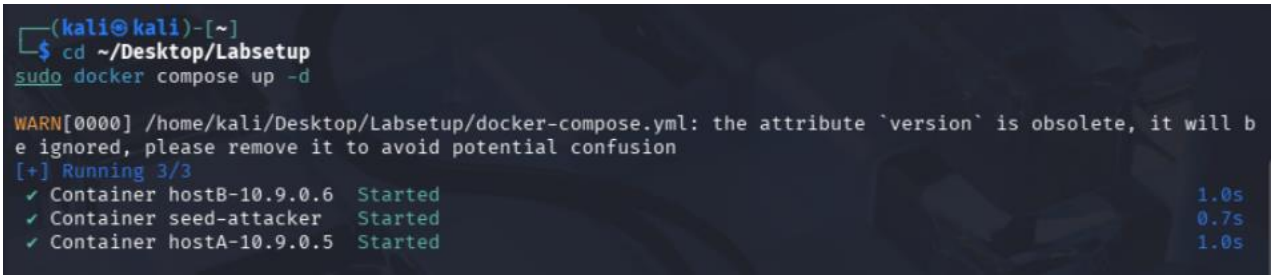
Machine	IP Address	Role
seed-attacker	10.9.0.1	Our attacker machine
hostA	10.9.0.5	Victim machine 1
hostB	10.9.0.6	Victim machine 2

We used Docker containers to simulate the lab network. Since we worked on Kali Linux, we first installed Docker and then started the lab containers using the provided `docker-compose.yml` file. After starting the containers, we confirmed that all three machines were running and connected to the same virtual network.

The attacker container was configured with `network_mode: host`. This means the attacker container shares the same network namespace as the host machine. Because of this setting, our attacker machine could see all traffic on the LAN — not just its own packets. Normal containers can only see their own traffic.

To find the network interface name, we used `ifconfig` and looked for the interface with IP address `10.9.0.1`. The interface name was `br-90fe1db80792`. We used this interface name in all our programs.





```
┌──(kali㉿kali)-[~/Desktop/Labsetup]
└─$ ifconfig
br-90fe1db80792: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.9.0.1  netmask 255.255.255.0  broadcast 10.9.0.255
        inet6 fe80::28d9:5aff:febe:1e90  prefixlen 64  scopeid 0x20<link>
        ether 2a:d9:5a:be:1e:90  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

3. Task Set 1 — Using Scapy

3.1 Task 1.1A — Packet Sniffing with Root and Without Root

We wrote a simple sniffer program using Scapy. The program listens on the bridge interface and captures ICMP packets. For each captured packet, it prints all the details using `pkt.show()`.

We ran the sniffer on the attacker machine with root privilege. Then, from Host A, we sent ping packets to Host B. The sniffer successfully captured every ICMP echo request and echo reply. We could see the source IP, destination IP, TTL, ICMP type, sequence number, and other fields clearly in the output.

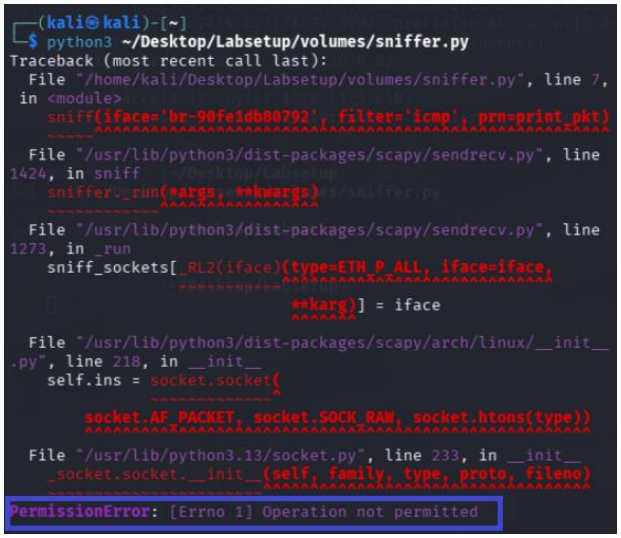
After that, we ran the same program without root privilege (as a normal user). This time, the program immediately failed and showed the error:

```
Operation not permitted
```

Observation: The program failed at the point where it tried to open the network interface with `pcap_open_live()`.

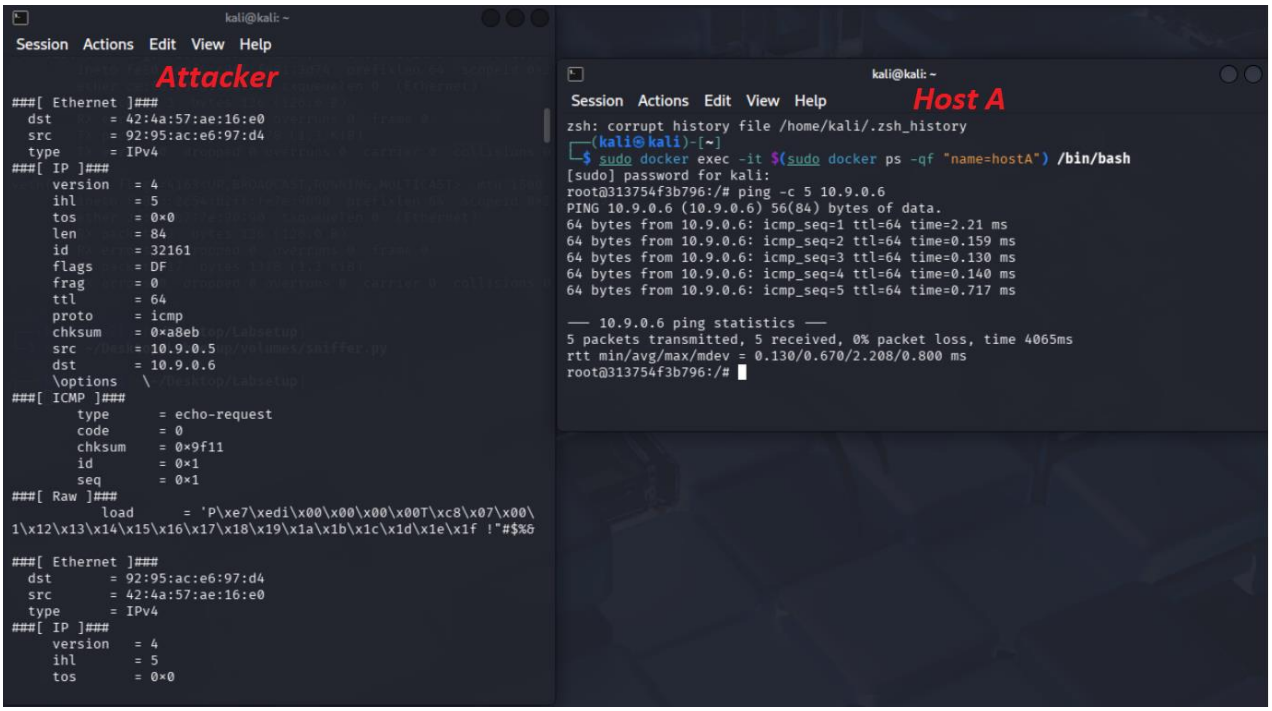
Explanation: To capture packets from a network interface, a program needs to put the network card into **promiscuous mode**. In normal mode, a network card only accepts packets that are addressed to its own MAC address. In promiscuous mode, it accepts all packets on the network, including packets meant for other machines.

Changing the network card to promiscuous mode requires kernel-level permission. The Linux kernel only allows this for processes running as root. This is an important security measure — if any regular user could activate promiscuous mode, they could spy on all network traffic without any restriction. The program fails exactly at `pcap_open_live()` because this is where the kernel permission is requested.



Error output without root — "Operation not permitted"

Sniffer output with root — showing captured ICMP packets



3.2 Task 1.1B — BPF Filters

We tested three different BPF (Berkeley Packet Filter) filters. BPF filters work at the kernel level — the kernel checks each packet before passing it to our program, so only matching packets reach our code. This is efficient because it avoids unnecessary processing.

Filter 1 — Only ICMP packets (filter='icmp')

We set the filter to capture only ICMP packets. From Host A, we sent pings and also made some TCP connections. Only the ICMP packets appeared in our sniffer output. All TCP and UDP traffic was completely invisible to our program.

Observation: The filter worked exactly as expected. Only ping traffic (ICMP echo request and reply) was captured. This shows how BPF filters can be used to focus on specific traffic types.

Sniffer output with icmp filter

```
sniff(iface='br-90fe1db80792', filter='icmp', prn=print_pkt)
```

```
kali@kali: ~
Session  Actions  Edit  View  Help
        chksum    = 0xb951
        id        = 0x2
        seq       = 0x5
###[ Raw ]###
          load      = '\xbe\xeb\xedi\x00\x00\x00\x00\xcc~\x07\x
00\x00\x00\x00\x00\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x
1b\x1c\x1d\x1e\x1f !"#$%&\'()*+,-./01234567'

###[ Ethernet ]###
  dst       = 92:95:ac:e6:97:d4
  src       = 42:4a:57:ae:16:e0
  type      = IPv4
###[ IP ]###
     version   = 4
     ihl       = 5
     tos       = 0x0
     len       = 84
     id        = 63671
     flags     =
     frag      = 0
     ttl       = 64
     proto     = icmp
     chksum    = 0x6dd5
     src       = 10.9.0.6
     dst       = 10.9.0.5
     \options   \
###[ ICMP ]###
        type      = echo-reply
        code      = 0
        chksum    = 0xc151
        id        = 0x2
        seq       = 0x5
###[ Raw ]###
          load      = '\xbe\xeb\xedi\x00\x00\x00\x00\xcc~\x07\x
00\x00\x00\x00\x00\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x
1b\x1c\x1d\x1e\x1f !"#$%&\'()*+,-./01234567'
```

```
kali@kali: ~
Session  Actions  Edit  View  Help
root@313754f3b796:/#
root@313754f3b796:/#
root@313754f3b796:/#
root@313754f3b796:/#
root@313754f3b796:/#
root@313754f3b796:/#
root@313754f3b796:/# ping -c 5 10.9.0.6
PING 10.9.0.6 (10.9.0.6) 56(84) bytes of data.
64 bytes from 10.9.0.6: icmp_seq=1 ttl=64 time=1.11 ms
64 bytes from 10.9.0.6: icmp_seq=2 ttl=64 time=0.184 ms
64 bytes from 10.9.0.6: icmp_seq=3 ttl=64 time=0.560 ms
64 bytes from 10.9.0.6: icmp_seq=4 ttl=64 time=0.139 ms
64 bytes from 10.9.0.6: icmp_seq=5 ttl=64 time=0.130 ms

--- 10.9.0.6 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4075ms
rtt min/avg/max/mdev = 0.130/0.425/1.114/0.379 ms
root@313754f3b796:/#
```

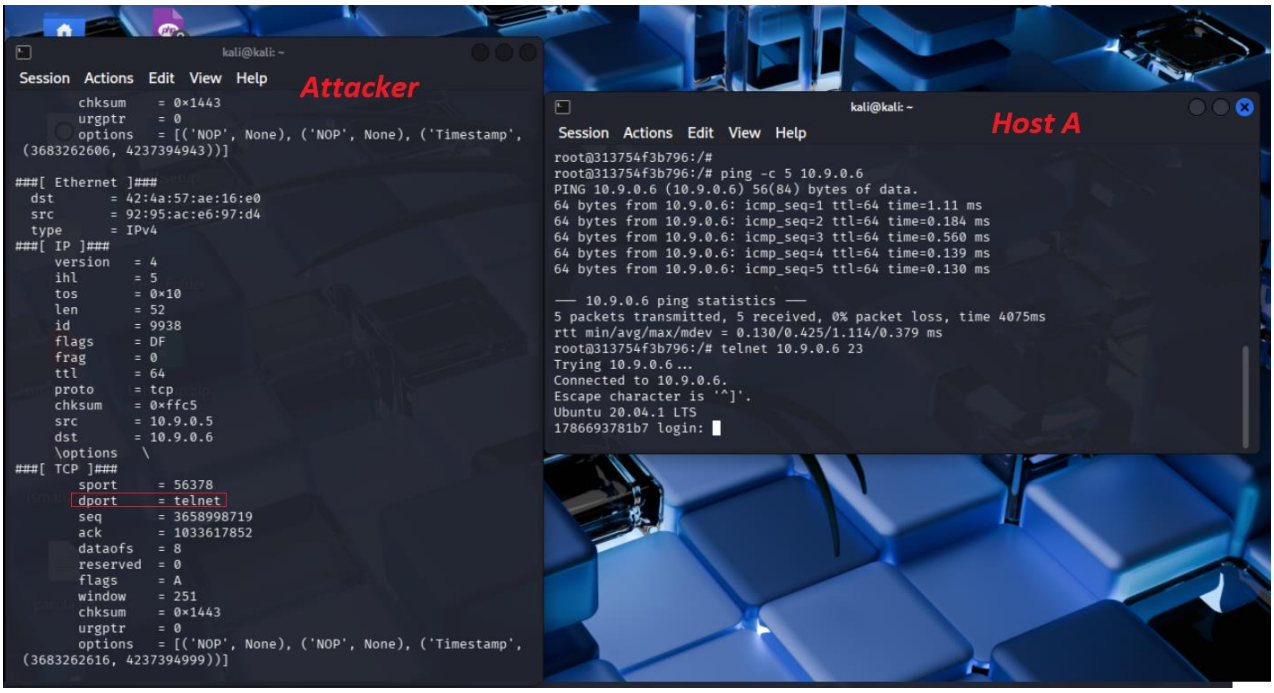
Filter 2 — TCP packets from a specific IP to port 23 (filter='tcp and src host 10.9.0.5 and dst port 23')

We set the filter to only capture TCP packets coming from 10.9.0.5 and going to port 23 (Telnet). From Host A, we connected to Host B using Telnet. The sniffer captured the Telnet packets.

Observation: All three conditions had to be true at the same time: TCP protocol, source IP 10.9.0.5, and destination port 23. Any traffic that did not match all three conditions was filtered out. We could also see that Telnet data is transmitted in plain text — each character the user typed appeared in the payload of a separate TCP packet. This is a serious security problem. SSH should always be used instead of Telnet because SSH encrypts all data.

Sniffer output with TCP port 23 filter showing Telnet packets

```
sniff(iface='br-90fe1db80792', filter='tcp and src host 10.9.0.5 and dst port 23', prn=print_pkt)
```

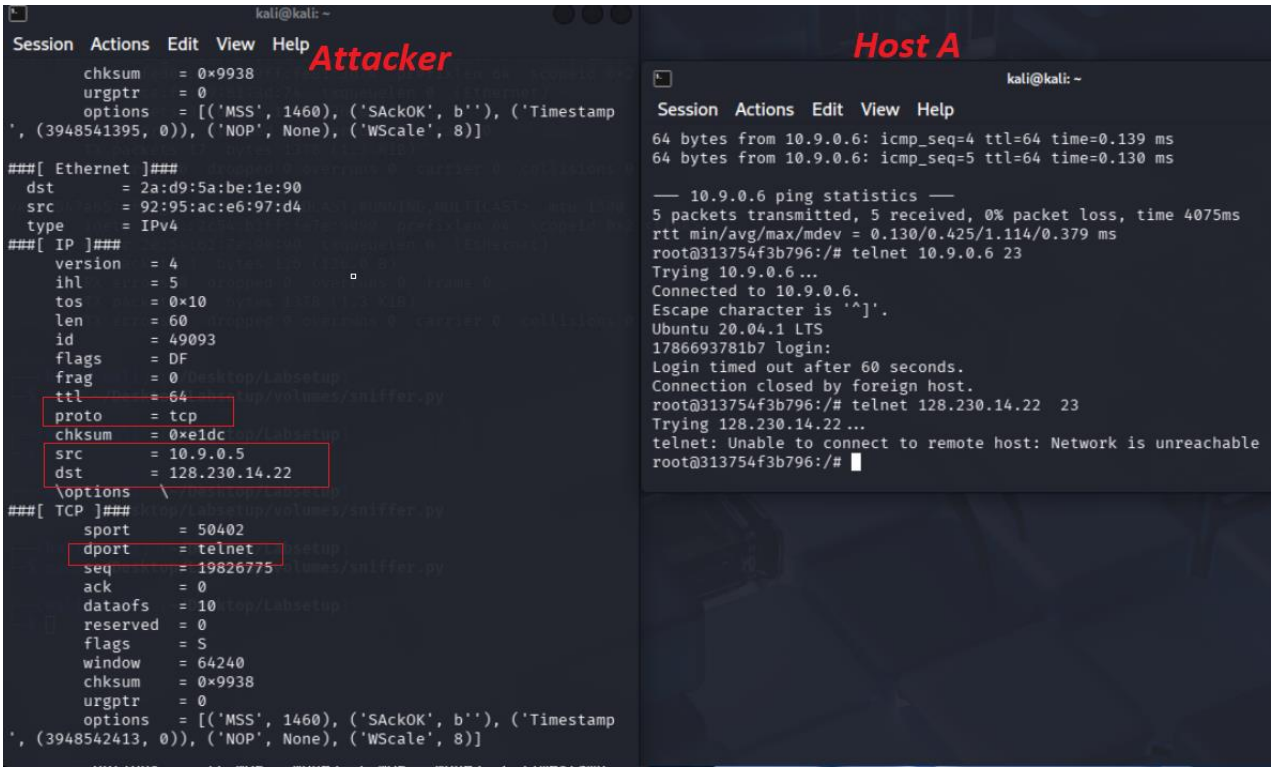


Filter 3 — Packets to or from a specific subnet (filter='net 128.230.0.0/16')

We set the filter to capture packets going to or coming from the 128.230.0.0/16 subnet. We sent a ping from Host A to an IP address in that subnet. Only those packets were captured.

Observation: The filter worked correctly. Traffic to other subnets was invisible. This type of filter is useful when monitoring specific network ranges for suspicious activity.

Sniffer output with subnet filter



Filter 3 was set to net 128.230.0.0/16. We attempted a telnet connection to 128.230.14.22 (which belongs to this subnet). Although the connection failed with "Network is unreachable", the sniffer successfully captured the outgoing TCP SYN packet destined for 128.230.14.22. This confirms that the filter correctly matches packets going to or coming from the 128.230.0.0/16 subnet.

3.3 Task 1.2 — Spoofing ICMP Packets

We wrote a spoofing program using Scapy. The program creates an ICMP echo request packet with a fake source IP address (`1.2.3.4`) and sends it to Host A (`10.9.0.5`). We opened Wireshark to observe the result.

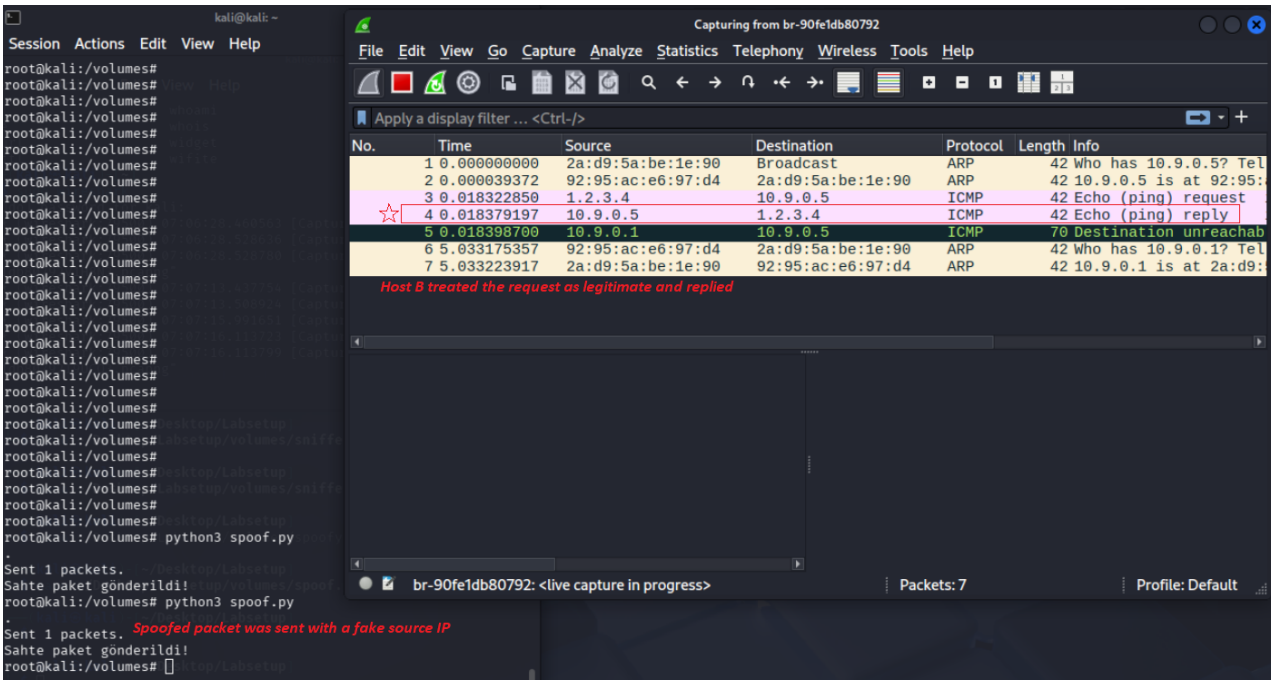
The program sends a packet where the source IP field is set to `1.2.3.4`, which does not exist anywhere on the Internet. In Scapy, the `/` operator is used to stack protocol layers — `IP()/ICMP()` creates a complete ICMP packet.

Observation: Wireshark showed two packets. First, an ICMP echo request from `1.2.3.4` to `10.9.0.5`. Then, an ICMP echo reply from `10.9.0.5` back to `1.2.3.4`. Host A was completely deceived — it replied to a fake address.

Explanation: The IP protocol does not have any mechanism to verify the source IP address. When Host A received the packet, it simply read the source IP field and sent a reply to that address. It had no way to know that `1.2.3.4` was a fake address. The reply packet was then lost on the network because no machine actually has that IP.

This is a fundamental design weakness of the IP protocol. In real attacks, this technique is used for DDoS reflection attacks: an attacker uses the victim's IP as the source address and sends requests to many large servers. All the servers reply to the victim, flooding it with traffic it never requested.

Wireshark showing the spoofed request and the reply from Host A



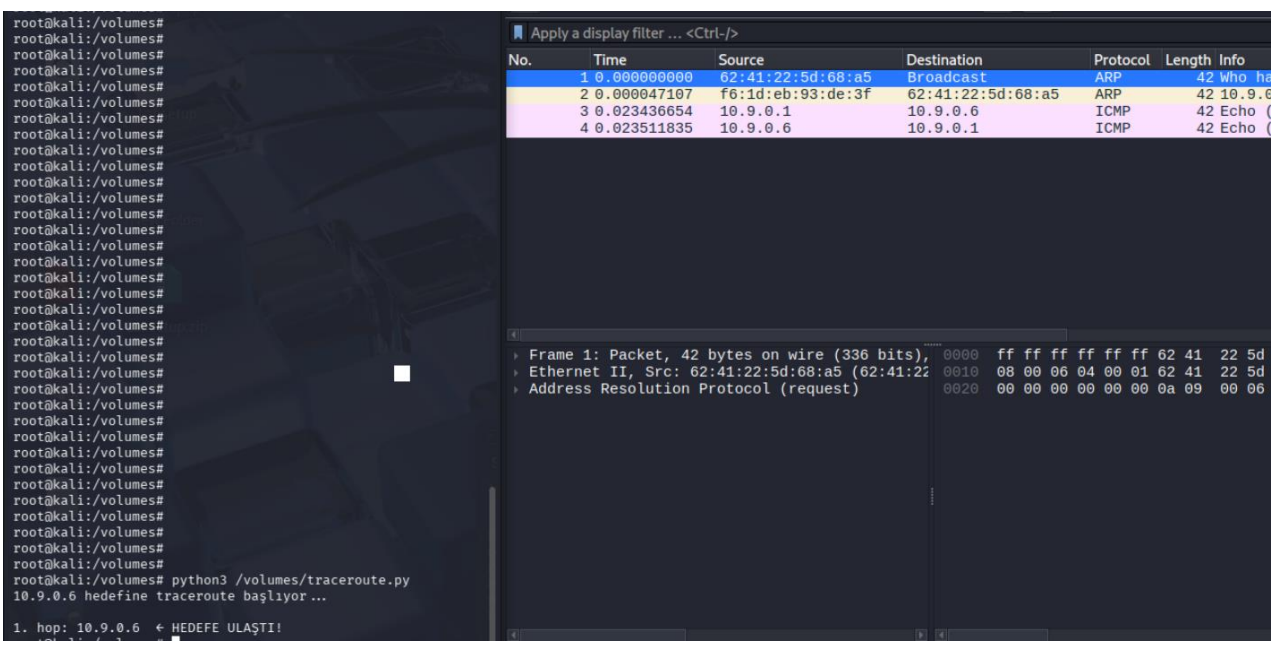
No.	What Happened	Explanation
1–2	ARP Request	Host B queried the MAC address of 10.9.0.5
3	ICMP Echo Request	1.2.3.4 → 10.9.0.5 ✅ Spoofed packet was sent with a fake source IP
4	ICMP Echo Reply	10.9.0.5 → 1.2.3.4 ✅ Host B treated the request as legitimate and replied

3.4 Task 1.3 — Traceroute

We implemented our own traceroute tool using Scapy. The tool works by sending ICMP packets with increasing TTL values, starting from 1. At TTL=1, the first router drops the packet and sends back an ICMP "Time Exceeded" error — this error message contains the router's IP address. Then we increase TTL to 2 to find the second router, and so on.

The TTL mechanism was originally designed to prevent packets from circulating forever on the network. Every router decreases TTL by 1. When TTL becomes 0, the router drops the packet. The ICMP Time Exceeded message that is sent back allows us to discover each router along the path. The real `traceroute` command works exactly the same way.

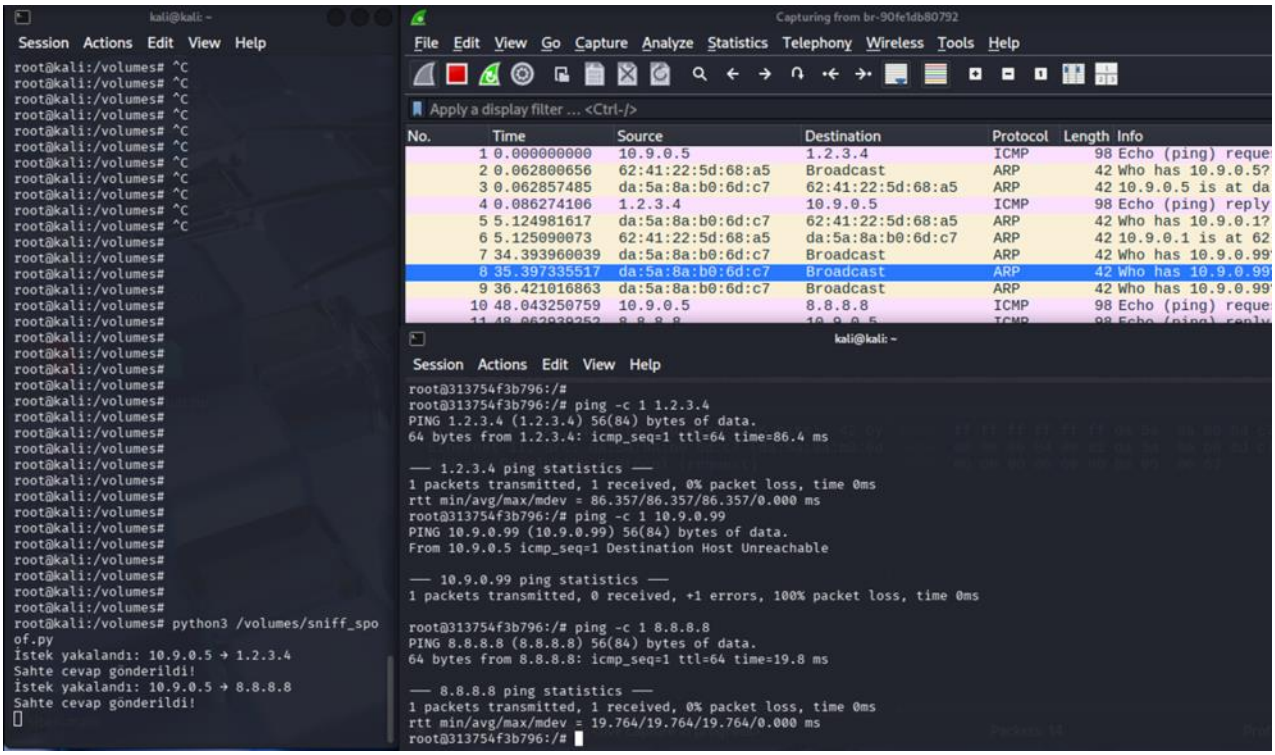
Traceroute output showing router hops



3.5 Task 1.4 — Sniff and then Spoof

We wrote a program that combines sniffing and spoofing. The program watches the network for ICMP echo requests. When it sees one, it immediately constructs a fake echo reply and sends it back. This makes the ping program on Host A receive replies even for IP addresses that do not exist.

We tested the program with three different targets from Host A:



ping 1.2.3.4 (non-existing host on the Internet)

When pinging 1.2.3.4, the ICMP echo request was routed through the default gateway since it is an internet address. Our sniffer captured the request on the LAN interface and immediately sent a spoofed echo reply. Host A received 1 received, 0% packet loss, confirming the spoofed reply was accepted.

ping 10.9.0.99 (non-existing host on the LAN)

When pinging 10.9.0.99, the kernel first sent an ARP request to resolve the MAC address of this LAN IP. Since no machine with this address exists, no ARP reply was received and the ICMP packet was never sent to the network interface. Therefore, our sniffer could not capture any request, and Host A received Destination Host Unreachable.

ping 8.8.8.8 (existing host on the Internet)

When pinging 8.8.8.8, both the real reply from Google's server and our spoofed reply were sent back to Host A. Host A received 1 received, 0% packet loss. This demonstrates that the sniff-and-spoof program works regardless of whether the destination host is alive or not, as long as the ICMP packet reaches the LAN interface.

4. Task Set 2 — Writing Programs in C

4.1 Task 2.1A — Packet Sniffer in C

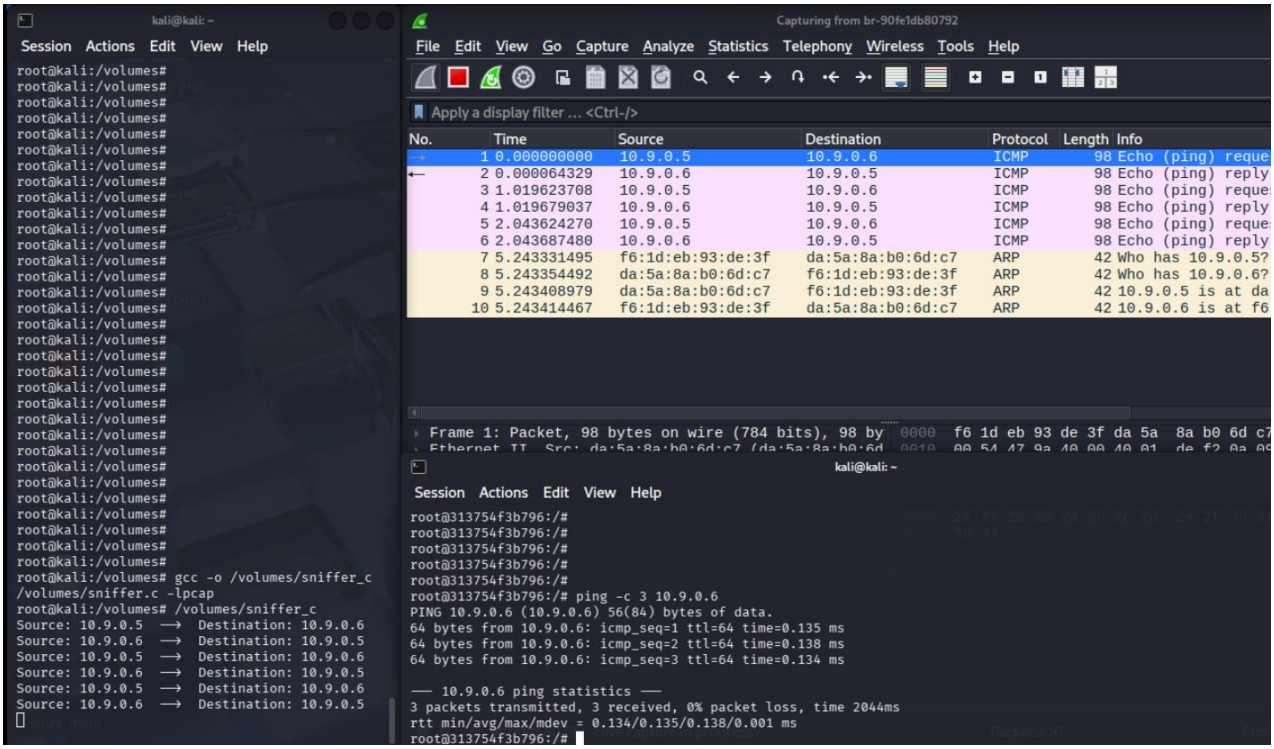
We wrote a packet sniffer program in C using the pcap library. The program opens the bridge interface, applies an ICMP filter, and prints the source and destination IP address of every captured packet.

The pcap library uses a callback model: we define a function `got_packet()` and tell pcap to call this function every time a packet is captured. We had to manually parse the raw packet bytes — the Ethernet header is 14 bytes, so the IP header starts at byte 14. We used `inet_ntoa()` to convert the binary IP addresses to readable strings.

We compiled the program with `gcc -o sniff sniff.c -lpcap` (the `-lpcap` flag links the pcap library). After running it and sending pings from Host A, the program successfully printed the source and destination IP addresses of each packet.

To sum up, sniffer program was compiled inside the container using gcc and libpcap. When Host A (10.9.0.5) pinged Host B (10.9.0.6), the sniffer successfully captured both the ICMP echo request and echo reply packets, printing the source and destination IP addresses for each. This confirms that the pcap library correctly intercepts all ICMP traffic on the bridge interface.

C sniffer output showing IP addresses



Question 1: Sequence of library calls essential for sniffer programs

A pcap-based sniffer follows four essential steps. First, pcap_open_live() opens the network interface for live packet capture. Second, pcap_compile() compiles the filter expression into a BPF program. Third, pcap_setfilter() applies the compiled filter to the capture handle. Finally, pcap_loop() starts the capture loop, invoking a user-defined callback function for each packet received.

Question 2: Why root privilege is needed? Where does it fail without root?

Root privilege is required because capturing packets from a network interface requires placing the network card into promiscuous mode, which is a kernel-level operation. Without root, the program fails at the pcap_open_live() call, which internally invokes pcap_create() and attempts to set the interface to promiscuous mode. The kernel rejects this operation for unprivileged users and returns an "Operation not permitted" error.

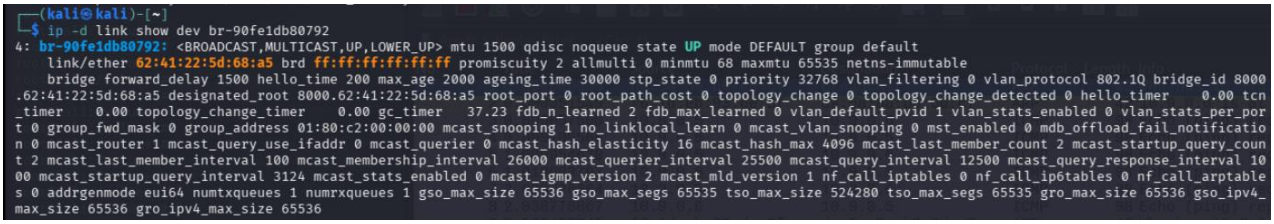
Question 3: Promiscuous mode ON vs OFF — how to demonstrate the difference?

When promiscuous mode is enabled (third parameter = 1 in pcap_open_live()), the sniffer captures all packets on the network, including those not destined for the attacker machine. When it is disabled

(parameter = 0), the sniffer only captures packets addressed to its own IP. To demonstrate this, we can run the sniffer on the attacker machine and ping between Host A and Host B. With promiscuous mode ON, the attacker captures those packets. With promiscuous mode OFF, no packets appear since none are addressed to the attacker.

This experiment clearly demonstrated the purpose of promiscuous mode and why it requires root privilege. Without it, sniffing other machines' traffic would be impossible.

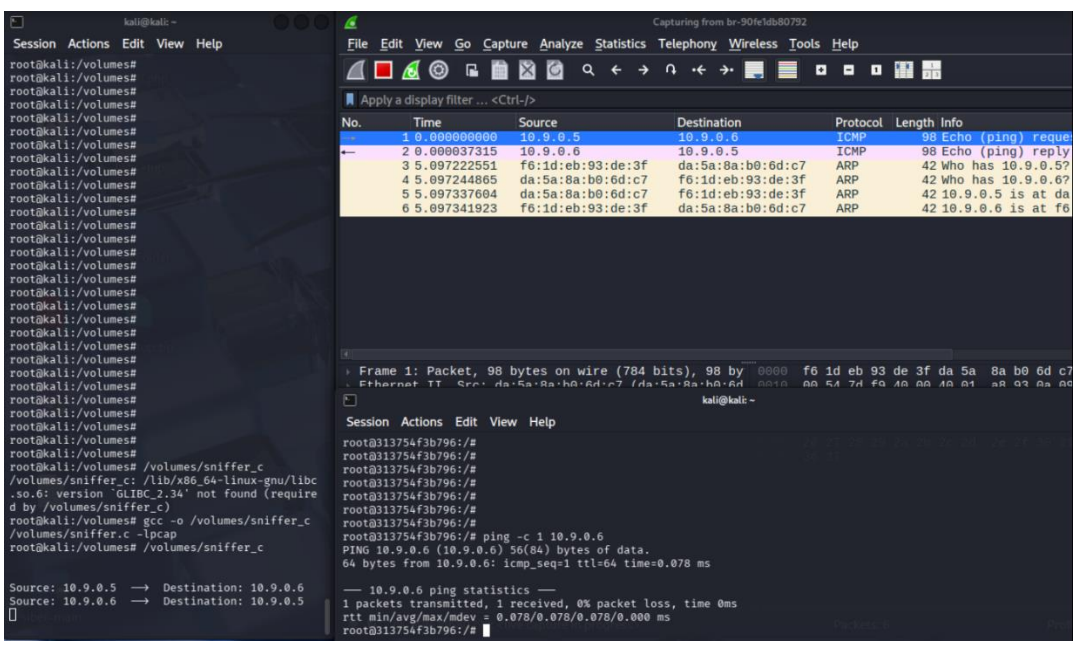
ip link output showing promiscuity 1



4.2 Task 2.1B — Writing Filters in C

Filter 1 — ICMP between two specific hosts

We used the filter `icmp and host 10.9.0.5 and host 10.9.0.6`. After recompiling and testing, only ICMP traffic between these two specific IP addresses was captured. Ping traffic involving any other IP was ignored.

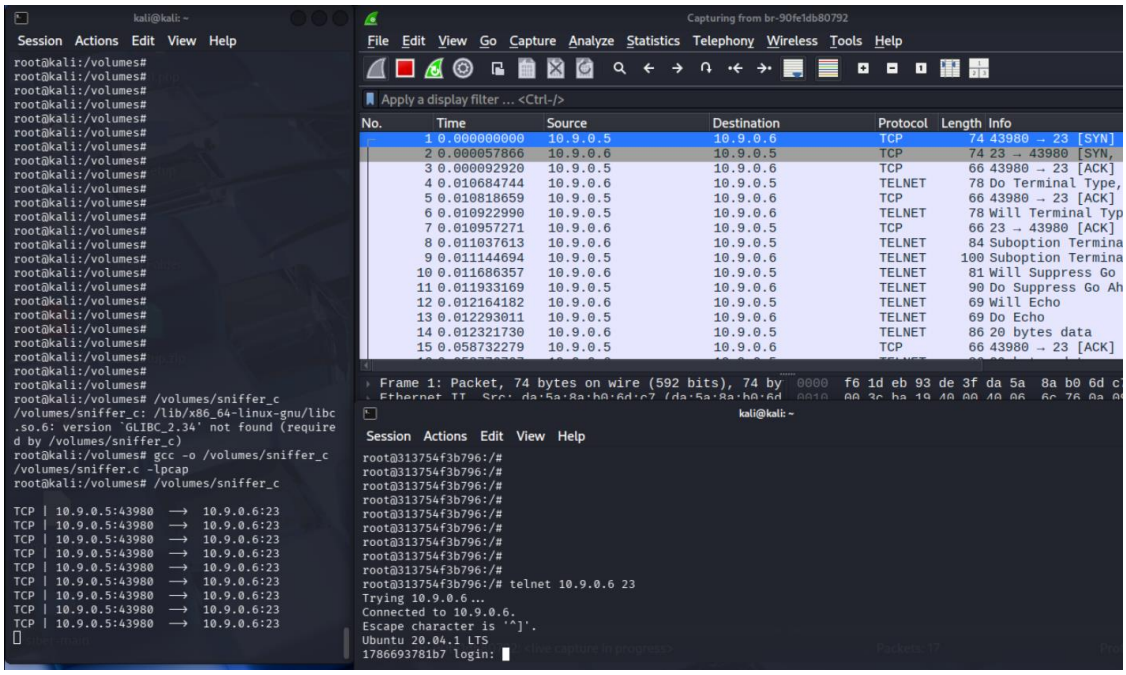


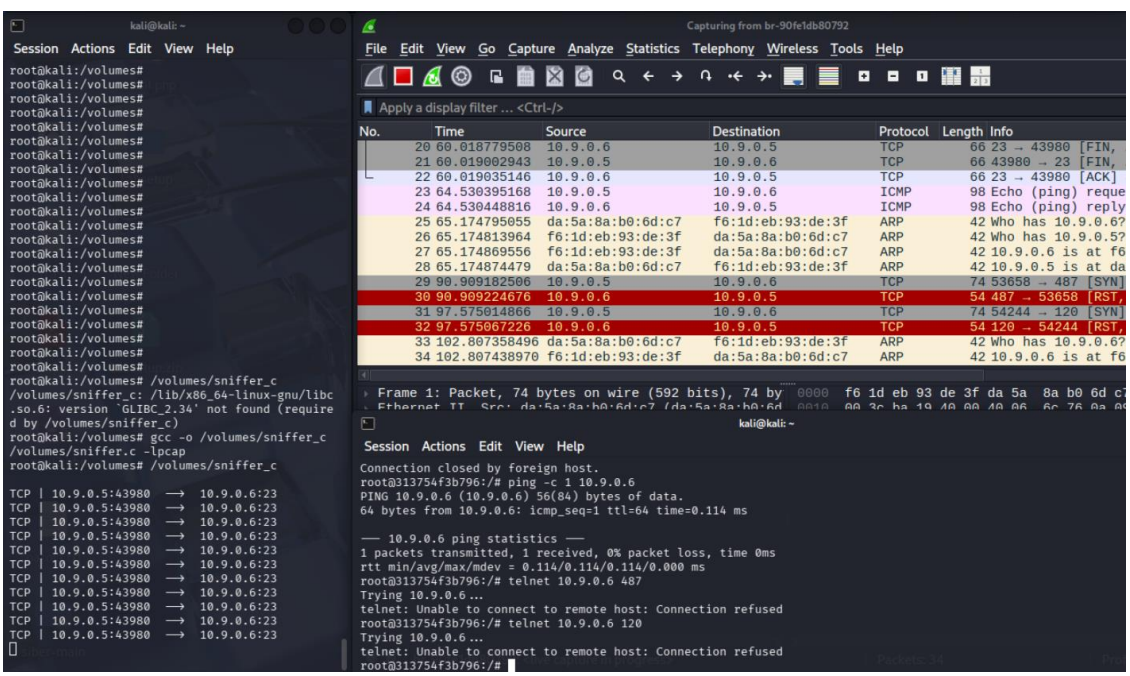
C sniffer with two-host ICMP filter

Filter 2 — TCP packets with destination port range 10-100

We used the filter `tcp and dst portrange 10-100`. When we made a Telnet connection (port 23) from Host A to Host B, the packets were captured because port 23 is within the range 10-100. Other TCP traffic on ports outside this range was not captured.

C sniffer with port range filter showing Telnet packets



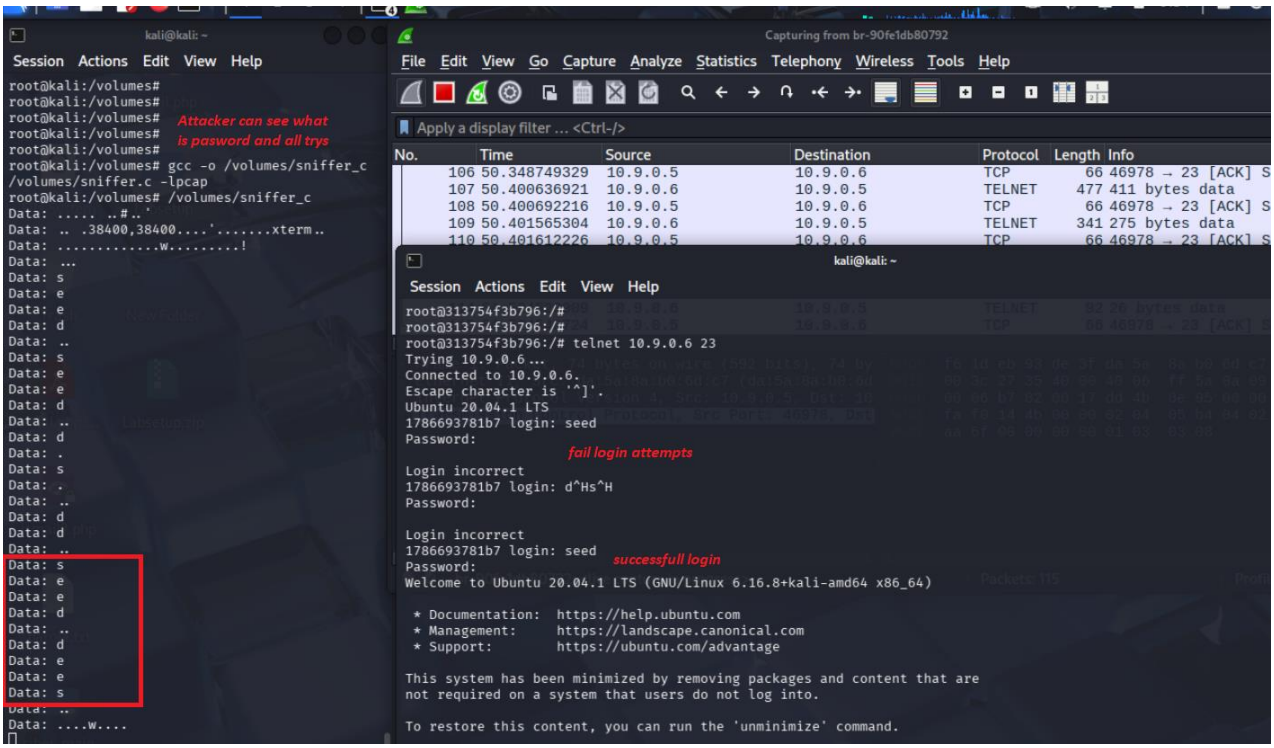


4.3 Task 2.1C — Sniffing Passwords

We modified the C sniffer to also print the data payload of each TCP packet. After setting the filter to capture Telnet traffic (TCP port 23), we ran the sniffer and then made a Telnet connection from Host A to Host B, typing a username and password.

Observation: We could see the characters typed by the user appearing in the sniffer output, one or a few characters at a time. The username and password were clearly visible in plain text.

Explanation: Telnet sends every keystroke immediately as a tiny TCP packet. This means the sniffer can reconstruct everything the user typed. The password was fully readable. This demonstrates the danger of using unencrypted protocols on a network. If this were a real corporate network and an attacker had compromised one machine, they could capture credentials from any Telnet session on the network. The solution is to always use SSH, which encrypts all traffic so that even captured packets are unreadable.

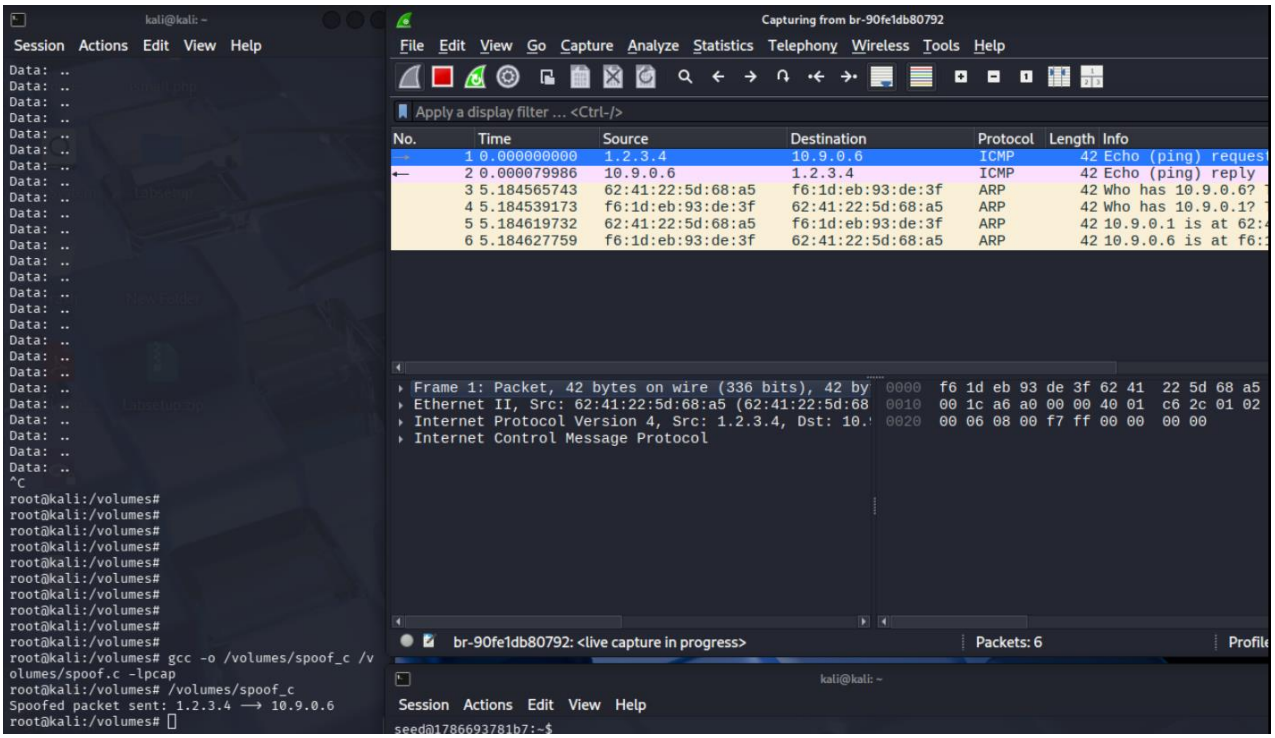


4.4 Task 2.2 — Packet Spoofing in C

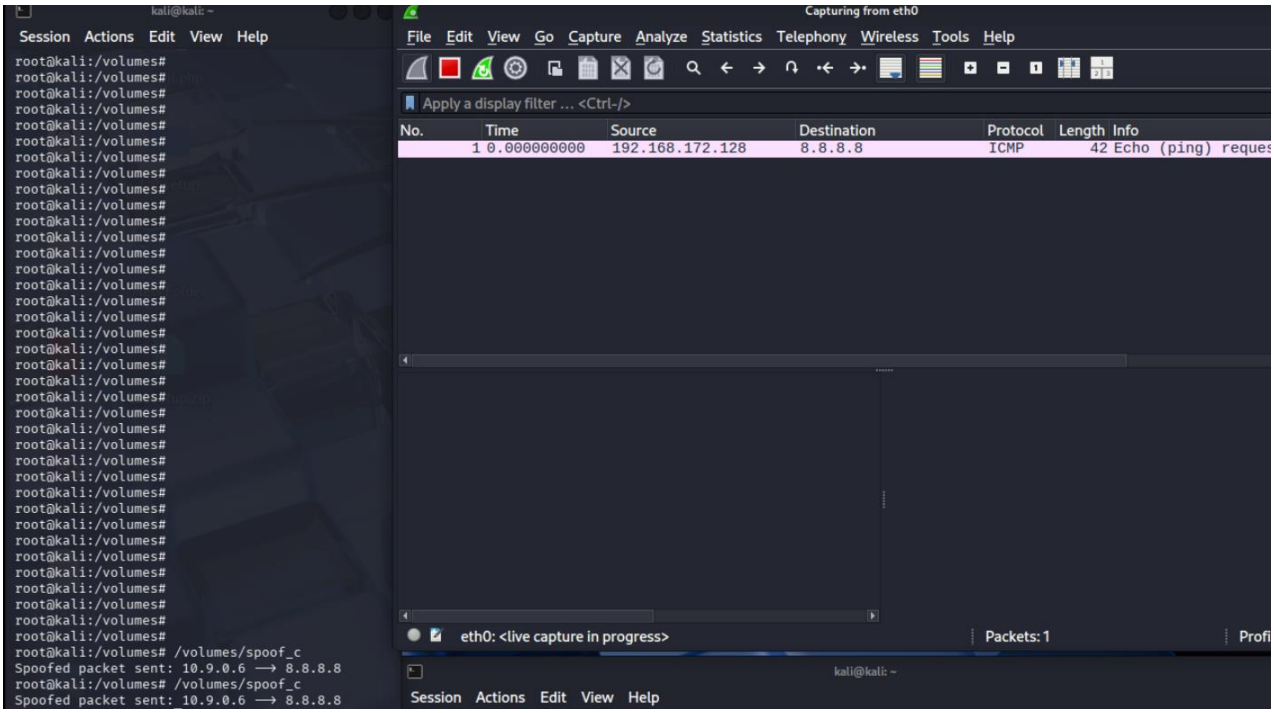
We wrote a spoofing program in C using raw sockets. With a raw socket created using `socket(AF_INET, SOCK_RAW, IPPROTO_RAW)`, we can construct the entire IP header ourselves and set any value we want — including a fake source IP address.

We manually built the IP header and ICMP header in a buffer. An important detail is the checksum: the Linux kernel automatically calculates the IP checksum when using `IPPROTO_RAW`, but the ICMP checksum must be calculated manually. We wrote a checksum function for this. If the ICMP checksum is wrong, the receiver silently discards the packet.

After sending the packet, Wireshark showed the spoofed request going from `1.2.3.4` to `10.9.0.5` and the reply coming back from Host A to `1.2.3.4`. The spoofing was successful.



Using raw sockets in C, we constructed an ICMP echo request packet with a spoofed source IP address (1.2.3.4) and sent it to Host B (10.9.0.6). Wireshark confirmed that the packet was received by Host B, which responded with an echo reply directed to the spoofed address. This demonstrates that raw sockets allow complete control over IP header fields, including the source address.



When the spoofed packet with source IP 10.9.0.6 was sent toward 8.8.8.8, the VMware NAT gateway replaced the source IP with its own address (192.168.172.128) before forwarding the packet to the internet. This is expected behavior — NAT translates internal IP addresses, which prevents the spoofed source address from reaching the external network. The spoofing was successful within the local network, as demonstrated in Task 2.2A.

Question 4: Can you set the IP packet length field to an arbitrary value?

Yes, the tot_len field in the IP header can be set to any arbitrary value using raw sockets. However, if the value is larger than the actual packet size, the receiving end may read garbage data beyond the real payload. If it is smaller, part of the payload will be ignored. The kernel does not validate this field before sending — it trusts the value provided by the programmer.

Question 5: Do you have to calculate the checksum for the IP header?

No. When using IPPROTO_RAW with the IP_HDRINCL socket option, the Linux kernel automatically calculates and fills in the IP header checksum. However, the ICMP checksum must be calculated manually by the programmer, since the kernel does not handle transport-layer checksums for raw sockets.

Question 6: Why do you need root privilege to run programs that use raw sockets? Where does it fail?

Raw sockets allow programs to craft arbitrary packets, including forging source IP addresses. This is a powerful and potentially dangerous capability, so the Linux kernel restricts raw socket creation to root users only. Without root, the program fails at the socket(AF_INET, SOCK_RAW, IPPROTO_RAW) call, returning a "Permission denied" error. This is the exact point where the kernel checks the process privileges.

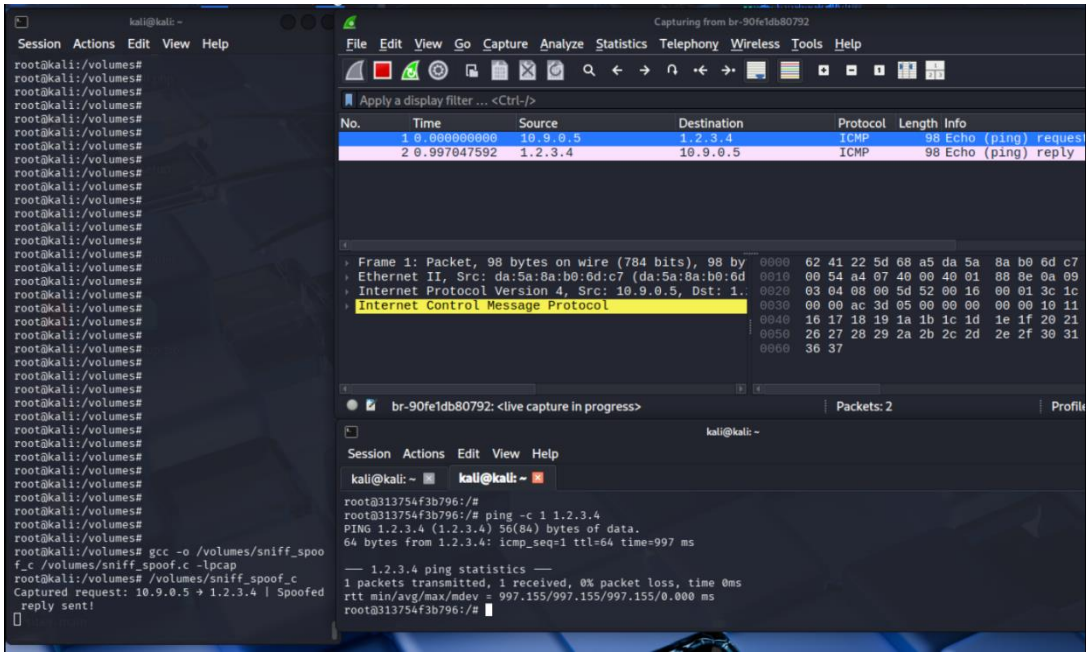
4.5 Task 2.3 — Sniff and then Spoof in C

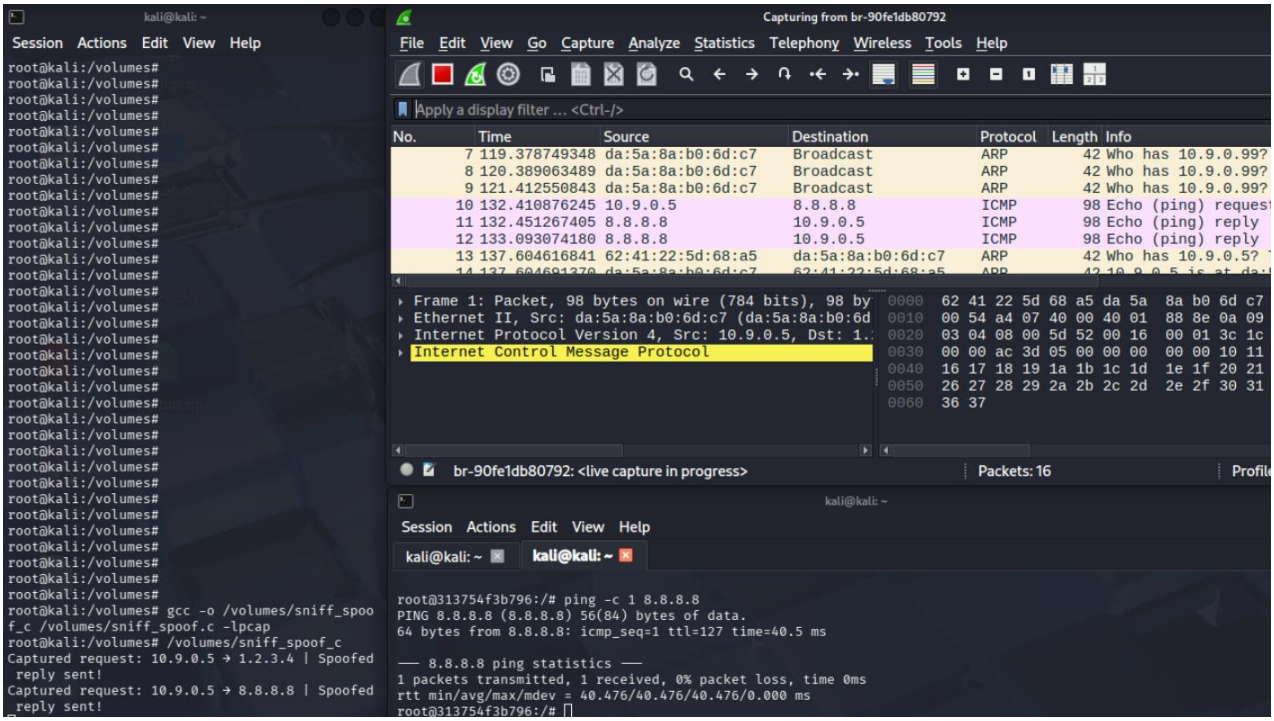
We combined sniffing (pcap) and spoofing (raw socket) in a single C program. The program uses pcap to watch for ICMP echo requests and immediately sends a fake echo reply using a raw socket.

We tested it with the same three IP addresses as in Task 1.4 (1.2.3.4, 10.9.0.99, and 8.8.8.8). The results were identical to the Scapy version:

- **1.2.3.4** — Host A received fake replies. The attack worked.
- **10.9.0.99** — No packets were captured. The attack failed because ARP could not resolve the LAN address.
- **8.8.8.8** — Host A received two replies per ping: one real and one fake, shown as duplicates.

Observation: The behavior was exactly the same as the Python/Scapy version. This confirms that Scapy is essentially a Python interface to the same underlying system calls (pcap and raw sockets). Writing the C version helped us understand what Scapy does internally.


kali@kali: ~
Session Actions Edit View Help
root@kali:/volumes#
root@kali:/volumes# gcc -o /volumes/sniff_spoof_c /volumes/sniff_spoof.c -lpcap
root@kali:/volumes# /volumes/sniff_spoof_c
Captured request: 10.9.0.5 → 1.2.3.4 | Spoofed reply sent!
Capturing from br-90fe1db80792
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1 0.000000000 10.9.0.5 1.2.3.4 ICMP 98 Echo (ping) request
2 0.997047592 1.2.3.4 10.9.0.5 ICMP 98 Echo (ping) reply
Frame 1: Packet, 98 bytes on wire (784 bits), 98 by
Ethernet II, Src: da:5a:8a:b0:6d:c7 (da:5a:8a:b0:6d
Internet Protocol Version 4, Src: 10.9.0.5, Dst: 1.
Internet Control Message Protocol
0000 62 41 22 5d 68 a5 da 5a 8a b0 6d c7
0010 00 54 a4 07 40 00 40 01 88 8e 0a 09
0020 03 04 08 00 5d 52 00 16 00 01 3c 1c
0030 00 00 ac 3d 05 00 00 00 00 00 10 11
0040 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21
0050 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31
0060 36 37
br-90fe1db80792: <live capture in progress>
Packets: 2
kali@kali: ~
Session Actions Edit View Help
root@313754f3b796:/#
root@313754f3b796:/# ping -c 1 1.2.3.4
PING 1.2.3.4 (1.2.3.4) 56(84) bytes of data.
64 bytes from 1.2.3.4: icmp_seq=1 ttl=64 time=997 ms
— 1.2.3.4 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 997.155/997.155/997.155/0.000 ms
root@313754f3b796:/#


kali@kali: ~
Session Actions Edit View Help
root@kali:/volumes# gcc -o /volumes/sniff_spoof_c /volumes/sniff_spoof.c -lpcap
root@kali:/volumes# /volumes/sniff_spoof_c
Captured request: 10.9.0.5 → 1.2.3.4 | Spoofed reply sent!
Captured request: 10.9.0.5 → 8.8.8.8 | Spoofed reply sent!
Capturing from br-90fe1db80792
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
7 119.378749348 da:5a:8a:b0:6d:c7 Broadcast ARP 42 Who has 10.9.0.99?
8 120.389063489 da:5a:8a:b0:6d:c7 Broadcast ARP 42 Who has 10.9.0.99?
9 121.412550843 da:5a:8a:b0:6d:c7 Broadcast ARP 42 Who has 10.9.0.99?
10 132.410876245 10.9.0.5 8.8.8.8 ICMP 98 Echo (ping) request
11 132.451267405 8.8.8.8 10.9.0.5 ICMP 98 Echo (ping) reply
12 133.093074180 8.8.8.8 10.9.0.5 ICMP 98 Echo (ping) reply
13 137.604616841 62:41:22:5d:68:a5 da:5a:8a:b0:6d:c7 ARP 42 Who has 10.9.0.5?
Frame 1: Packet, 98 bytes on wire (784 bits), 98 by
Ethernet II, Src: da:5a:8a:b0:6d:c7 (da:5a:8a:b0:6d
Internet Protocol Version 4, Src: 10.9.0.5, Dst: 1.
Internet Control Message Protocol
0000 62 41 22 5d 68 a5 da 5a 8a b0 6d c7
0010 00 54 a4 07 40 00 40 01 88 8e 0a 09
0020 03 04 08 00 5d 52 00 16 00 01 3c 1c
0030 00 00 ac 3d 05 00 00 00 00 00 10 11
0040 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21
0050 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31
0060 36 37
br-90fe1db80792: <live capture in progress>
Packets: 16
kali@kali: ~
Session Actions Edit View Help
root@313754f3b796:/# ping -c 1 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=127 time=40.5 ms
— 8.8.8.8 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 40.476/40.476/40.476/0.000 ms
root@313754f3b796:/#

5. Discussion

Why Sniffing is Dangerous

Our experiments showed that sniffing is simple to perform if an attacker can get a machine on the same network. With just a few lines of code and root privilege, every packet on the LAN becomes readable. The Telnet password capture experiment was especially revealing — a user typing their password on Host A had no idea that we were reading every character on the attacker machine.

The main protection against sniffing is encryption. If all data is encrypted before it is sent, a sniffer can capture the packets but cannot read the content. Protocols like HTTPS, SSH, and TLS provide this protection. Network segmentation (VLANs, subnets) and switch-based networks (instead of hubs) also reduce the scope of what a sniffer can see — but they do not fully eliminate the threat.

Why Spoofing is Dangerous

Our experiments confirmed that the IP protocol has no mechanism to verify source addresses. Anyone with root privilege can send packets pretending to be any IP address. This is not a bug — it is a fundamental design of the Internet. The original designers prioritized simplicity and performance over security.

Spoofing enables many attack types. In DDoS reflection and amplification attacks, the attacker uses spoofing to make large volumes of traffic be directed at a victim. In some cases, attackers also use spoofing to bypass IP-based access controls. Modern defenses include BCP38 (ISPs filtering traffic with spoofed source addresses) and stateful firewalls.

The Role of ARP

Task 1.4 showed an important limitation of network-level attacks: the ARP protocol. For two machines on the same LAN to communicate, they need each other's MAC addresses. If an IP address has no corresponding MAC address (because the machine does not exist), the packet is never sent. This is why our sniff-and-spoof attack failed for the local IP 10.9.0.99. Understanding ARP is essential for understanding how LAN-based attacks work and why some attacks require additional techniques (like ARP spoofing) to succeed.

6. Conclusion

This lab gave us a deep, practical understanding of packet sniffing and packet spoofing — two foundational techniques in network security.

By writing our own programs at two different levels (Scapy in Python and pcap/raw sockets in C), we learned not only how to perform these attacks but also why they work. The IP protocol's lack of source address verification makes spoofing trivially easy. The ability to put network cards into promiscuous mode makes sniffing straightforward for anyone with root access to a machine on the network.

The most important takeaways for network security:

- **Encryption is the only real defense against sniffing.** Even if packets are captured, encrypted data cannot be read.
- **IP source addresses cannot be trusted.** Any security system that relies solely on IP addresses for authentication is vulnerable.
- **Unencrypted protocols (Telnet, FTP, HTTP) should not be used** on any network where security matters.
- **Understanding how attacks work at a low level** is necessary for designing effective defenses.

Overall, this lab reinforced the importance of building security into network protocols and applications from the ground up, rather than relying on assumptions about the network environment.